



Aprendizado por Reforço

Aula 8: Programação Dinâmica

Prof. Charles Madeira

Contexto

- A Programação Dinâmica (PD) refere-se a uma coleção de métodos de otimização que podem ser usados para a descoberta de políticas ótimas, dado um modelo perfeito do ambiente (função de transição T) como um processo de decisão de Markov (MDP)
- Embora os métodos clássicos de PD sejam de utilidade limitada no Aprendizado por Reforço (AR), devido à suposição de um modelo perfeito e ao alto custo computacional exigido, eles são importantes teoricamente pois fornecem a base necessária para a área
- Todos os métodos de AR podem ser vistos como tentativas de alcançar um efeito similar ao da PD, mas necessitando de menor poder computacional e sem assumir um modelo perfeito do ambiente

Contexto

- A ideia base da PD é a **decomposição do problema original em subproblemas**
 - Subestrutura ótima
 - Princípio da otimalidade
 - Solução ótima pode ser decomposta em subproblemas
 - Subproblemas que se sobrepõem
 - Subproblemas podem ocorrer várias vezes
 - Soluções podem ser guardadas e reutilizadas

Divisão e Conquista



Armazenamento



Dependência de Soluções



Programação Dinâmica

Caracterização do problema

- Exemplo da série de Fibonacci

$$\text{FIB}(N) = \begin{cases} \text{FIB}[N - 1] + \text{FIB}[N - 2], & \text{se } N > 1 \\ 0, & \text{se } N = 0 \\ 1, & \text{se } N = 1 \end{cases} \quad 0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

- Encontrar o termo N da sequência de Fibonacci é um problema que pode ser resolvido com Programação Dinâmica

Caracterização do problema

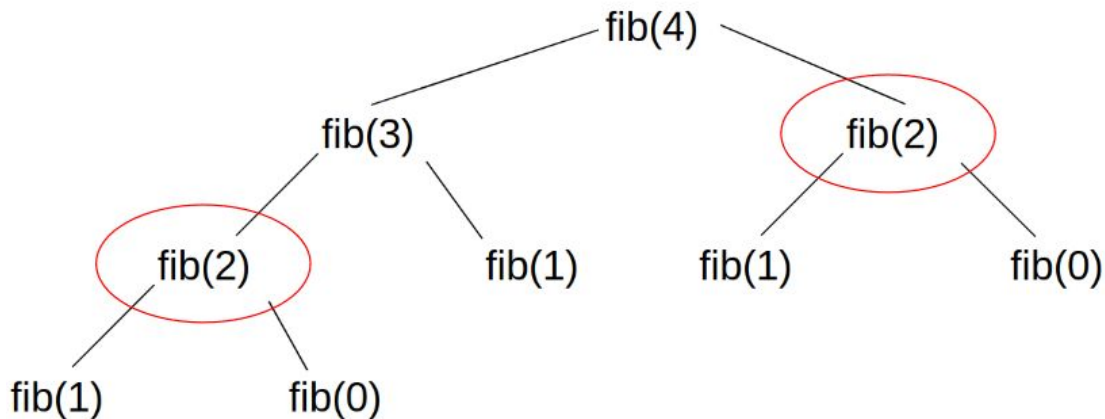
- **Etapa 1:** Identificar o problema como sendo de Programação Dinâmica

$$\text{FIB}(N) = \begin{cases} \text{FIB}[N - 1] + \text{FIB}[N - 2], & \text{se } N > 1 \\ 0, & \text{se } N = 0 \\ 1, & \text{se } N = 1 \end{cases}$$

- **Subestrutura ótima:** a solução ótima do problema pode ser obtida a partir das soluções dos subproblemas dependentes

Caracterização do problema

- **Etapa 1:** Identificar o problema como sendo de Programação Dinâmica



- **Sobreposição de soluções:** a solução ótima do problema passa pela resolução de subproblemas que aparecem duas ou mais vezes

Caracterização do problema

- **Etapa 2:** Definir o valor da solução ótima recursivamente
 - Para o caso da série de Fibonacci, esta etapa é bastante simples, uma vez que a própria função que encontra a solução do termo N é demandada novamente para descoberta dos termos $N-1$ e $N-2$

```
def fibonacci(n):  
    if n == 1 or n == 0:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)
```

Abordagens das soluções

- **Etapa 3:** Calcular o valor da solução ótima da forma “top-down” ou “bottom-up”
 - **Top-down** (memorização)
 - Parte da solução geral ótima que se deseja encontrar e segue analisando quais subproblemas devem ser resolvidos até alcançar um subproblema com resolução trivial

$\text{fib}(N) \rightarrow \text{fib}(N-1) \rightarrow \text{fib}(N-2) \rightarrow \dots \rightarrow \text{fib}(2) \rightarrow \text{fib}(1) \rightarrow \text{fib}(0)$

- Os resultados são armazenados em uma tabela e são reutilizados quando o mesmo subproblema aparece novamente

Abordagens das soluções

- **Etapa 3:** Calcular o valor da solução ótima da forma “top-down” ou “bottom-up”
 - **Top-down** (memorização)

```
def fibonacciTopDown(n, table = {}):  
    if n == 1 or n == 0:  
        return n  
    try:  
        return table[n]  
    except:  
        table[n] = fibonacciTopDown(n-1) + fibonacciTopDown(n-2)  
        return table[n]
```

Solução Trivial

Busca na tabela

Armazena resultado

Abordagens das soluções

- **Etapla 3:** Calcular o valor da solução ótima da forma “top-down” ou “bottom-up”
 - Bottom-up (tabulação)
 - Parte da solução trivial e desenvolve os cálculos até encontrar a solução ótima do problema

$\text{fib}(0) \rightarrow \text{fib}(1) \rightarrow \text{fib}(2) \rightarrow \dots \rightarrow \text{fib}(N-2) \rightarrow \text{fib}(N-1) \rightarrow \text{fib}(N)$

- Os resultados são armazenados em uma tabela e são reutilizados quando o mesmo subproblema aparece novamente

Abordagens das soluções

- **Etapa 3:** Calcular o valor da solução ótima da forma “top-down” ou “bottom-up”
 - Bottom-up (tabulação)

```
def fibonacciBottomUp(n, table = {}):  
    table[0] = 0  
    table[1] = 1  
    for cont in range(2, n + 1):  
        table[cont] = table[cont - 1] + table[cont - 2]  
    return table[n]
```

Solução Trivial

Busca na tabela e armazena

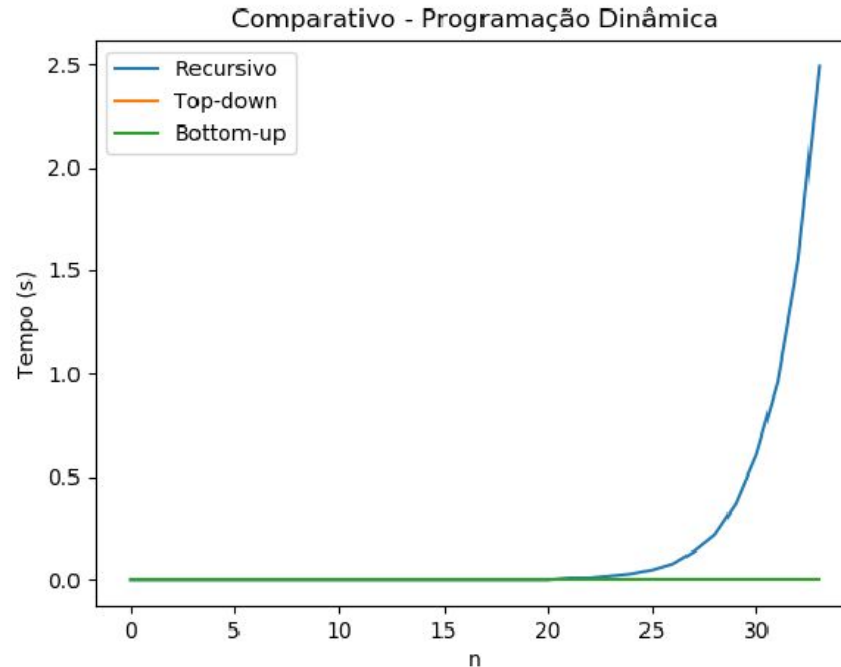
Comparativo entre as abordagens

- Top-down x Bottom-up

Quesitos	Top-down	Bottom-up
Especificação do problema	Mais fácil de pensar no problema	Formular o problema pode ser complexo
Código	Codificação mais simples e com regras mais simples	Inviável quando há muitas condições
Resolução dos subproblemas	Resolve apenas os subproblemas que são necessários	Resolve todos os subproblemas do problema original
Parâmetros na tabela	Tabela preenchida apenas com as soluções necessárias	Preenche a tabela com soluções que podem ser desnecessárias

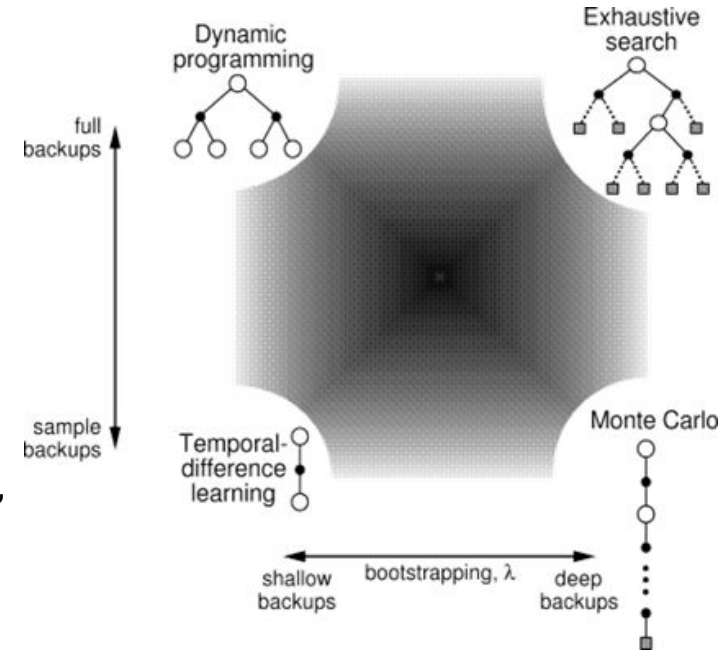
Análise de Desempenho

- Recursivo x Top-down x Bottom-up



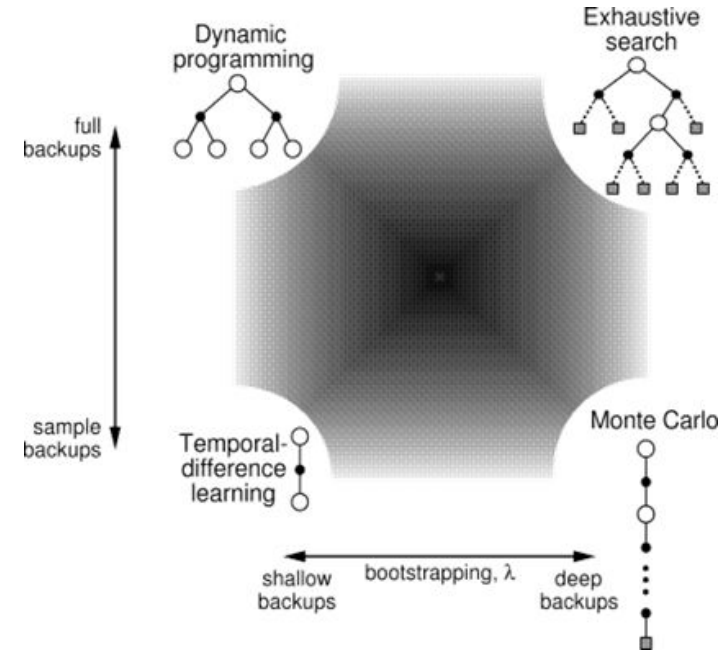
Métodos base para o Aprendizado por Reforço

- Busca Exaustiva (força bruta)
 - Estimativa de todos os possíveis candidatos de uma solução
 - Simplicidade e garantia de otimalidade
 - Custo extremo devido à explosão combinatória
- Programação Dinâmica (DP)
 - Reduz o custo em relação à busca exaustiva, mas necessita de um modelo perfeito do ambiente, o que limita a sua utilidade em casos de problemas complexos
 - É teoricamente importante para o Aprendizado por Reforço, pois fornece a base para a compreensão de todos os seus métodos
 - Os métodos de aprendizado por reforço podem ser vistos adaptações da DP, eliminando a exigência de um modelo perfeito do ambiente e reduzindo o custo computacional



Métodos base para o Aprendizado por Reforço

- Monte Carlo (MC)
 - Estimativa da função de valor com base nas experiências obtidas por meio da interação com o ambiente
 - Valores obtidos a partir do cálculo da média dos retornos (reforço) ao final de cada episódio
- Diferenças Temporais (DT)
 - Emprego de técnicas de bootstrapping (estimativas intermediárias) da função de valor para estimar o retorno (reforço) de cada episódio
 - Busca associar as abordagens visando aproveitar o melhor dos dois mundos:
 - Aspecto incremental da Programação Dinâmica
 - Recurso à experiência do Monte Carlo



Planejamento com a Programação Dinâmica

- Requer conhecimento prévio completo do funcionamento do ambiente
 - Isso raramente é obtido no contexto de problemas reais
- Usado para planejamento em um MDP, nos modos de predição e controle
 - Predição
 - Entrada:
 - MDP (S, A, T, R, γ)
 - Política π
 - Saída:
 - Função valor V^π
 - Controle
 - Entrada: MDP (S, A, T, R, γ)
 - Saída:
 - Função valor ótima V^*
 - Política ótima π^*

Exercício

- Experimentar a Programação Dinâmica no Frozen Lake
 - Grid 4x4
 - Determinística
 - Estocástica
- Usar como base a implementação disponível no [Drive](#) da disciplina

